

DEVELOPER ONBOARDING

Kommon School Backend Guide

A Beginner-Friendly Walkthrough — Built for a Fresher Joining the Team

19

MODULES

25+

DB TABLES

6

CRON JOBS

5

RETRY
ATTEMPTS

Node.js 20 · Express · Prisma · PostgreSQL · Redis · BullMQ · Razorpay
Generated on 2026-05-22 · For: sumagodev1@gmail.com

Table of Contents

1. What is Kommon School?
2. The Big Picture — Architecture
3. Tech Stack & Why Each Tool
4. Project Folder Structure
5. How the Server Starts (boot order)
6. Configuration & Environment Vars
7. Database Schema Deep Dive
8. The Golden Path — Student Enrollment Journey
9. Authentication & JWT Tokens
10. RBAC — Roles & Permissions
11. Enrollment Module Deep Dive
12. Payment Module Deep Dive
13. Razorpay Webhook Handling
14. External API Sync (Sumago)
15. BullMQ Queues & Workers
16. Cron Jobs & Distributed Locks
17. Admin Modules Overview
18. Middleware Stack
19. API Response Pattern
20. Error Handling Strategy
21. Important Edge Cases
22. Local Development Setup
23. Glossary of Terms

1. What is Kommon School?

Kommon School Backend is the brain behind a student enrollment + online payment platform. It handles the entire journey: from a student filling out an "Enroll Now" form on the website, to paying through Razorpay, to being provisioned into a third-party learning platform called **Sumago**.

Explain Like I'm 5 (ELI5)

Imagine a coaching class that takes online admissions. A student visits the website, fills a form, pays the fees online, and gets login credentials to start learning. This backend is the kitchen behind that restaurant — students see a clean menu (frontend), but behind the scenes, this backend cooks the meal: it stores the form data, takes the money safely, calls Razorpay, waits for confirmation, and then tells another system (Sumago)

to give the student their textbooks. It also keeps shopkeeper-side tools (admin panel) so the team can manage admissions, payments, follow-ups, and reports.

What problems does it solve?

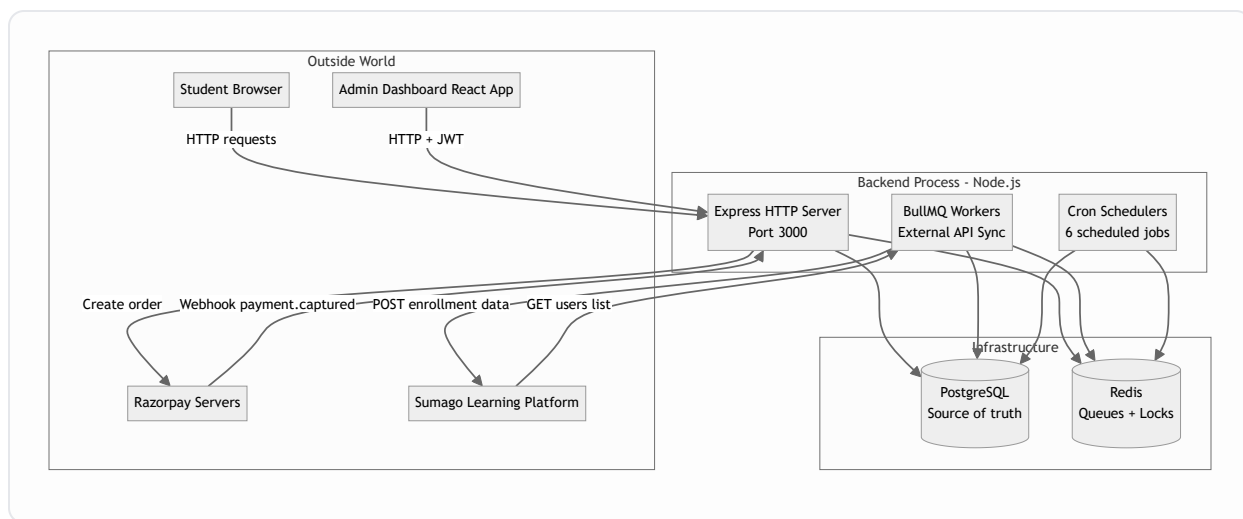
- **Take online payments safely** using Razorpay (no money should be lost, ever).
- **Handle thousands of simultaneous enrollments** without creating duplicate records or duplicate charges.
- **Provision students automatically** on the Sumago learning platform after successful payment.
- **Retry automatically** if any step fails (network outage, Sumago down, etc.) — no manual babysitting.
- **Give admins control** via dashboards: see enrollments, payments, follow-ups, reports, retry failed syncs.
- **Track everything for audit** — every privileged action is logged.

Two kinds of enrollments

Type	Who creates it?	How?	candidate_type
EXTERNAL	The student themselves	Public "Enroll Now" form on the website + Razorpay payment	EXTERNAL
INTERNAL	Admin/Marketing team	Manually one-by-one OR bulk CSV upload from the admin dashboard. Uses internal plans & coupons.	INTERNAL

2. The Big Picture — Architecture

The system isn't just one Express server. It's a small ecosystem with multiple moving parts that talk to each other through PostgreSQL and Redis.



Explain Like I'm 5 (ELI5)

Think of the backend like a busy post office.

- The **Express server** is the front counter where customers (students, admins) come and submit forms.
- **PostgreSQL** is the big filing cabinet where all permanent records live.
- **Redis** is the sticky-notes board — fast, temporary, used to remember "this job needs doing" or "I'm currently working on this, don't touch it".
- **BullMQ workers** are the postmen who go out and deliver letters (call Sumago's API) in the background. They retry if a delivery fails.
- **Cron jobs** are the night-shift clerks who clean up stale paperwork at fixed schedules.

Why this design?

- **Decouple slow work from fast responses.** When a student pays, the user-facing response returns in milliseconds. The slow part (calling Sumago, retrying on failure) happens in a worker after the response is sent.

- **Survive Sumago outages.** If Sumago is down, jobs queue up in Redis and retry later instead of failing the student's enrollment.
- **Scale horizontally.** You can run multiple Express+Worker instances on different servers. Redis distributed locks make sure cron jobs only run on ONE instance at a time.

3. Tech Stack & Why Each Tool

Tool	Purpose	Why?
Node.js 20+	JavaScript runtime	Modern features, native test runner, fast.
Express 4	HTTP framework	Industry standard, huge ecosystem, simple middleware model.
Prisma 5	ORM (Object-Relational Mapper)	Type-safe queries, automatic migrations, easy schema management.
PostgreSQL 15	Database	Strong consistency, ACID transactions, row-level locking (for race protection).
Redis 7	In-memory store	Job queue backend + distributed locks + ephemeral caching.
BullMQ	Background job queue	Reliable retries with exponential backoff, deduplication by jobId.
node-cron	Scheduled jobs	Run code at fixed intervals (every 5 min, daily 2 AM, etc.).
Razorpay SDK	Payment gateway	Indian payment processor — UPI, cards, netbanking.
Joi	Input validation	Validate request bodies, query params, route params with declarative schemas.
jsonwebtoken	JWT tokens	Stateless authentication, supports HS256 (dev) and RS256 (prod).
bcrypt	Password hashing	Slow on purpose — makes brute-force attacks impractical.
helmet	HTTP security headers	Sets defaults like X-Content-Type-Options, etc.
winston	Structured logging	JSON logs with trace IDs — searchable in production.
opossum	Circuit breaker	Stop hammering Sumago when it's clearly down.
ioredis	Redis client	Robust, supports clustering & sentinel.

Tool	Purpose	Why?
multer	File uploads	Handles CSV uploads for admin bulk enrollment.
csv-parse / csv-stringify	CSV parsing & generation	Bulk enrollment imports & report exports.

```

common_school_backend/
├─ docs/                # Markdown docs (course master, OpenAPI spec)
├─ logs/                # Winston log output (rotated daily)
├─ scripts/             # One-off scripts (seeds, backfills)
├─ src/
│   ├─ app.js           # Express app: middleware + route registration
│   └─ server.js        # Boot: HTTP listen, queues, jobs, graceful
shutdown
├─
│   ├─ config/          # Singletons & validated env
│   │   ├─ env.js       # Reads .env, validates, exports typed config
│   │   ├─ database.js  # Prisma client singleton
│   │   ├─ redis.js     # ioredis singleton
│   │   ├─ jwt.js       # JWT sign/verify (HS256 or RS256)
│   │   ├─ logger.js    # Winston logger
│   │   └─ constants.js # HTTP codes, error codes, defaults
│   └─ middleware/      # Express middleware
│       ├─ auth.middleware.js    # Extract & verify Bearer token
│       ├─ rbac.middleware.js    # Role/permission checks
│       ├─ errorHandler.middleware.js # Catches all errors → standard envelope
│       ├─ requestLogger.middleware.js # Per-request logging with traceId
│       ├─ rateLimit.middleware.js  # IP-based rate limit (login, etc.)
│       ├─ validate.middleware.js   # Joi validation runner
│       └─ webhookRaw.middleware.js  # Capture raw body for HMAC
├─
└─ modules/             # One folder per feature
    ├─ auth/            # Login, refresh, logout, change-password
    ├─ enrollments/     # Public student enrollment + plan select
    ├─ payments/        # Razorpay order create + verify
    ├─ razorpay/        # Webhook handler from Razorpay
    ├─ webhooks/        # Sumago user-sync webhook delivery
    ├─ externalApi/     # Builds & sends payload to Sumago
    ├─ followups/       # Marketing followup lifecycle
    ├─ plans/           # Public + Admin plan/pricing CRUD
    ├─ internalPlans/   # Admin-only internal plan CRUD
    ├─ courses/         # Course master CRUD
    ├─ courseNameMaster/ # Course name lookup CRUD
    ├─ educationMaster/  # Education level master
    ├─ durationMaster/   # Duration label master
    ├─ promoCodes/      # Promo code validation
    └─ reports/         # Aggregations, CSV export

```

```

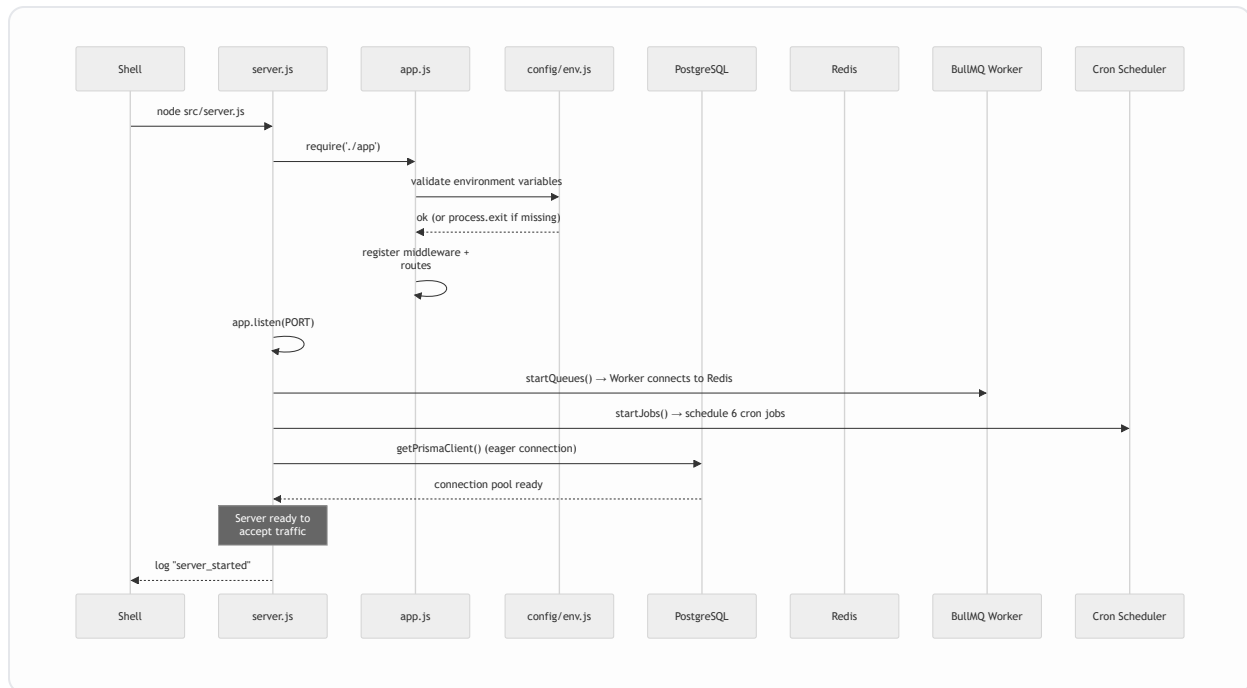
├─ permissions/           # Permission DB lookup + cache
├─ audit/                 # Audit log read + write
├─ adminEnrollments/      # Admin manual + bulk enrollment
├─ admin/                 # Group: superadmin-only modules
│   ├─ users/             # Admin user CRUD
│   ├─ razorpayConfigs/    # Encrypted Razorpay key mgmt
│   ├─ externalApiLogs/    # Read-only log viewer
│   ├─ dashboard/         # Dashboard counters
│   ├─ adminEnrollments/   # List view
│   ├─ adminPayments/      # List view
│   └─ adminFollowupsReport/ # Report view
├─ queues/                # BullMQ
│   ├─ connection.js       # Shared ioredis connection
│   ├─ externalApi.queue.js # Enqueue helpers
│   ├─ workers/externalApi.worker.js # Job processor
│   └─ index.js            # startQueues() / stopQueues()
├─ jobs/                  # node-cron scheduled jobs
│   ├─ paymentReconciliation.job.js # Every 5 min
│   ├─ externalApiRetry.job.js      # Every 2 min
│   ├─ webhookRetry.job.js          # Every 3 min
│   ├─ enrollmentCleanup.job.js     # Daily 2 AM
│   ├─ refreshTokenCleanup.job.js   # Daily 3 AM
│   ├─ followupReminder.job.js      # Daily 9 AM
│   └─ index.js                     # startJobs() / stopJobs()
├─ prisma/
│   ├─ schema.prisma             # Database schema (single source of truth)
│   ├─ seed.js                   # Seed superadmin + permissions
│   └─ migrations/               # Auto-generated SQL migration files
├─ tests/unit/                  # Pure in-process unit tests
├─ docs/                        # OpenAPI spec
├─ utils/                       # Shared helpers
│   ├─ ApiError.js              # Custom error class
│   ├─ ApiResponse.js           # sendSuccess / sendError
│   ├─ asyncHandler.js          # Wraps async controllers
│   ├─ crypto.js                # AES-256-GCM, HMAC, bcrypt
│   ├─ distributedLock.js        # Redis-based mutex
│   └─ pagination.js            # Parse page/limit/sort
├─ ecosystem.config.js          # PM2 cluster config (production)
├─ nginx.conf.sample            # Reverse proxy template
└─ package.json

```

Modules follow a consistent pattern: `routes.js` → `controller.js` → `service.js` → `repository.js`. Routes wire URLs to controllers; controllers parse requests; services hold business logic; repositories talk to Prisma. Validators sit beside controllers.

5. How the Server Starts (boot order)

When you run `npm start`, here's what happens, in order:



Graceful shutdown

When SIGTERM (Ctrl+C, PM2 reload, Docker stop) arrives, the server doesn't just die. It:

1. Stops accepting new HTTP requests.
2. Stops scheduling new cron ticks (in-flight ticks finish).
3. Drains BullMQ workers — lets active jobs complete (up to 15s).
4. Disconnects Prisma cleanly.
5. Disconnects Redis.
6. Exits cleanly (or forces exit after 15s timeout).

6. Configuration & Environment Variables

Every config value lives in `.env`. The file `src/config/env.js` validates them at boot — if a required var is missing, the server refuses to start.

Required for any environment

Variable	Purpose
<code>DATABASE_URL</code>	Postgres connection string
<code>JWT_ACCESS_SECRET</code>	32+ char secret for HS256 access tokens
<code>JWT_REFRESH_SECRET</code>	32+ char secret for HS256 refresh tokens
<code>RAZORPAY_KEY_ID</code>	Razorpay public key (seeded into DB)
<code>RAZORPAY_KEY_SECRET</code>	Razorpay secret (encrypted into DB)
<code>RAZORPAY_WEBHOOK_SECRET</code>	HMAC secret for webhook verification
<code>ENCRYPTION_MASTER_KEY</code>	32 bytes hex — encrypts secrets at rest
<code>SEED_ADMIN_EMAIL</code> / <code>SEED_ADMIN_PASSWORD</code>	First superadmin created on seed

Required from Phase 2A onwards

Variable	Default	Purpose
<code>REDIS_URL</code>	<code>redis://localhost:6379</code>	Connection for BullMQ + locks
<code>EXTERNAL_API_URL</code>	(webhook.site default)	Sumago provisioning endpoint
<code>EXTERNAL_API_TOKEN</code>	—	Bearer token for Sumago
<code>EXTERNAL_API_TIMEOUT_MS</code>	<code>15000</code>	Per-call HTTP timeout

Optional tuning knobs (Phase 2B)

Variable	Default	What it controls
<code>CRON_TIMEZONE</code>	<code>UTC</code>	Timezone for daily cron schedules
<code>RECONCILIATION_PENDING_GRACE_MS</code>	<code>600000</code>	Wait 10 min before checking Razorpay
<code>RECONCILIATION_EXPIRE_AFTER_MS</code>	<code>1800000</code>	Expire pending after 30 min
<code>ENROLLMENT_CLEANUP_STALE_HOURS</code>	<code>24</code>	Auto-expire submitted enrollments
<code>WEBHOOK_RETRY_MAX_ATTEMPTS</code>	<code>5</code>	Max replays per failed webhook

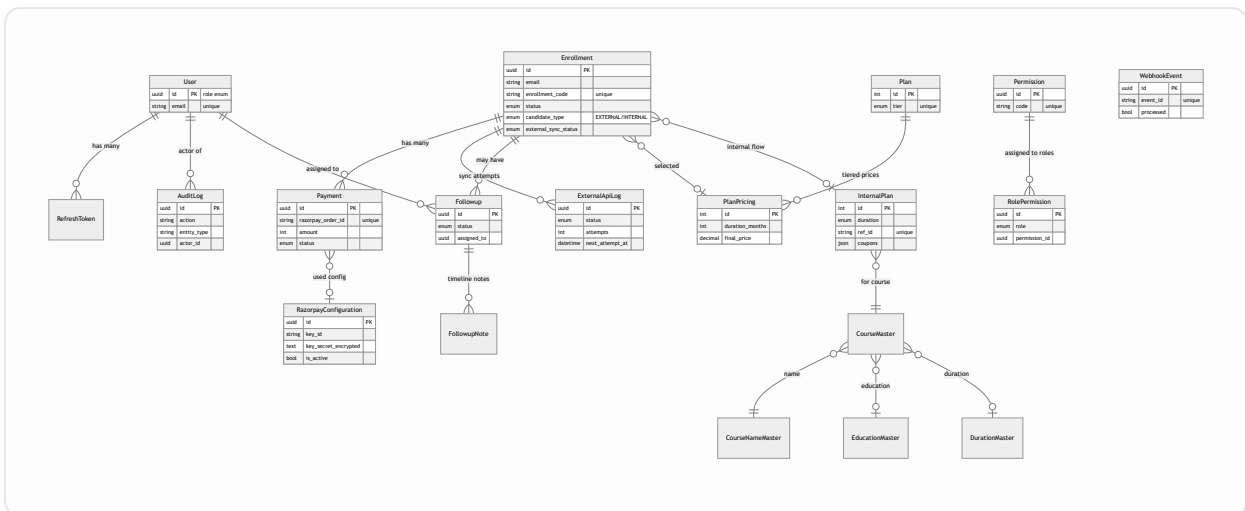
Warning

Never commit `.env` to git. The repo includes `.env.example` — copy it locally and fill in real secrets.

7. Database Schema Deep Dive

All schema is defined in `src/prisma/schema.prisma`. PostgreSQL is the source of truth — Redis is ephemeral, BullMQ jobs are recoverable from the DB.

Main entities



Important enums

EnrollmentStatus

`submitted` → `payment_pending` → `paid` / `completed`

On failure: `failed` or `expired`
Customer-facing payment lifecycle.

ExternalSyncStatus

`PENDING` → `SUCCESS` / `DEAD_LETTER`

Or stuck at `FAILED` while retrying.

Third-party (Sumago) sync state. **Separate from EnrollmentStatus** so the admin UI can show "paid but not yet provisioned".

PaymentStatus

`initiated` → `pending` → `success`

Or: `failed`, `cancelled`, `expired`, `refunded`

ExternalApiStatus (per log row)

`pending`, `processing`, `success`, `failed`, `retrying`, `dead_letter`

Tracked per attempt in
`external_api_logs` .

FollowupStatus

`payment_pending` , `call_back_later` ,
`interested` , `not_interested` ,
`payment_completed` , `followup_closed` ,
`invalid_number` , `no_response`
`payment_completed` &
`followup_closed` are terminal.

Role

`superadmin` (can do everything),
`admin` (manage data), `marketing`
(followups only)

Why both `candidate_type` and `external_sync_status` ?

Explain Like I'm 5 (ELI5)

Think of an enrollment as a parcel.

- `candidate_type` = WHERE the parcel came from (the website front door or the admin office).
- `EnrollmentStatus` = whether the parcel was paid for (the postage stamp).
- `external_sync_status` = whether the parcel was actually delivered to Sumago (the destination).

These are independent because the parcel can be paid for but stuck in transit to Sumago.

8. The Golden Path — Student Enrollment Journey

This is the most important flow in the entire system. Memorize it.



Syntax error in text
mermaid version 10.9.6

Explain Like I'm 5 (ELI5)

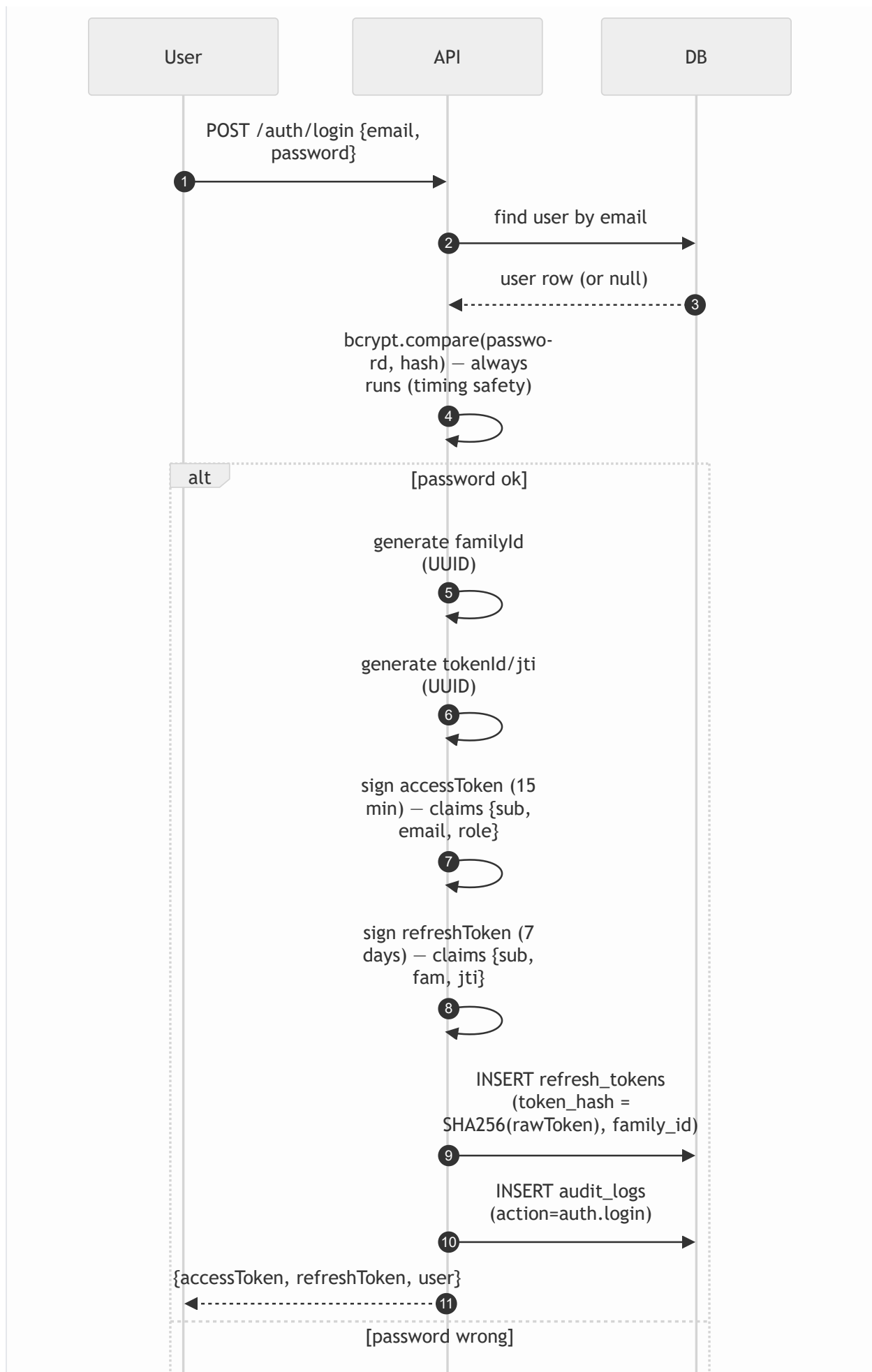
Read the diagram top to bottom like a story:

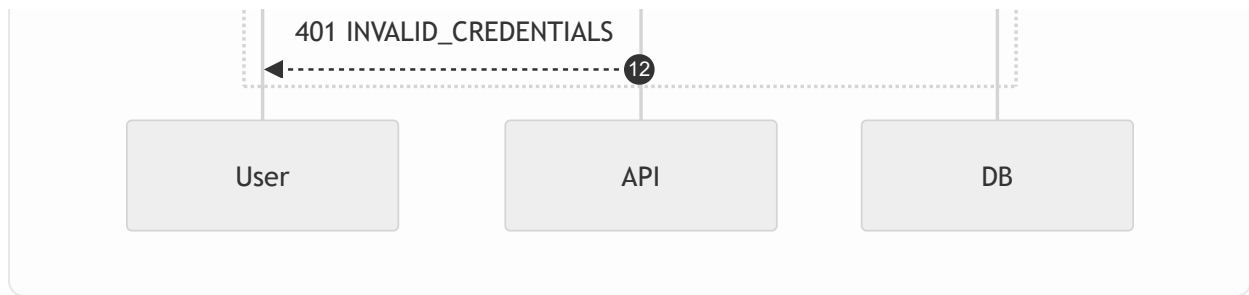
1. Student submits form → row created in DB.
2. Student picks plan → row updated with planPricingId.
3. Student clicks "Pay" → backend asks Razorpay for an order, stores it, returns the orderId.
4. Razorpay shows the payment popup. Student pays.
5. Razorpay sends payment success info back to the frontend. Frontend tells the backend "verify this".
6. Backend checks the signature (proof Razorpay sent it), marks the payment as paid, and queues a job to tell Sumago.
7. Worker picks up the job, calls Sumago. If it fails, retry up to 5 times. If still failing, mark dead-letter and create a marketing follow-up.
8. Meanwhile, Razorpay also sends a webhook to us directly — this is a safety net in case the frontend never called verify (e.g., browser crashed). The same race-protected code runs, whichever arrives first wins.

9. Authentication & JWT Tokens

`src/modules/auth/` + `src/config/jwt.js` + `src/middleware/auth.middleware.js`

How login works





Why two tokens?

- **Access token (15 min):** Sent on every request as `Authorization: Bearer ...`. Short-lived so a stolen one expires fast.
- **Refresh token (7 days):** Stored more carefully (httpOnly cookie or secure storage). Used only to get new access tokens.

Token rotation & reuse detection

Explain Like I'm 5 (ELI5)

Imagine your refresh token is a movie ticket stub. Every time you use it to get a new access token, the box office **punches a hole in it** (marks it revoked) and gives you a brand new stub. If anyone tries to use the punched stub again, the box office knows it's stolen and **cancels every ticket in your booking family** — you have to log in again from scratch. This is called "refresh token rotation with reuse detection". It catches stolen tokens.

Family tracking

Every login creates a new `family_id`. Every refresh keeps the same `family_id` but creates a new `jti`. If a token in a family is ever reused after revocation, all tokens in that family are revoked.

JWT signing algorithms

Mode	Algorithm	When to use
HS256	Symmetric (one secret)	Development, simple setups
RS256	Asymmetric (public/private keys)	Production — public key can be shared with services that need to verify but not sign

The code in `config/jwt.js` auto-picks based on `JWT_ALGORITHM` env var.

Endpoints

Method	Path	Auth	Purpose
POST	/api/v1/auth/login	Public	Email + password → tokens
POST	/api/v1/auth/refresh	Public	Refresh token → new pair
POST	/api/v1/auth/logout	Bearer	Revoke all user tokens
GET	/api/v1/auth/me	Bearer	Current user info
POST	/api/v1/auth/change-password	Bearer	Old + new pwd → revokes all tokens

10. RBAC — Roles & Permissions

Two layers of access control are stacked:

1. **Coarse role check** — `authorize(['admin', 'superadmin'])` from `middleware/rbac.middleware.js`
2. **Granular permission check** — `hasPermission('payments:retry')` using `permissions / role_permissions` tables

Default role × permission matrix

Permission code	superadmin	admin	marketing
<code>enrollments:view</code>	yes	yes	yes
<code>payments:view</code>	yes	yes	no
<code>payments:retry</code>	yes	no	yes
<code>followups:view</code>	yes	yes	yes
<code>followups:manage</code>	yes	no	yes
<code>razorpay_configs:manage</code>	yes	no	no
<code>users:manage</code>	yes	no	no
<code>reports:view</code>	yes	yes	no
<code>external_api_logs:view</code>	yes	yes	no
<code>audit_logs:view</code>	yes	yes	no

Pro Tip

Superadmin always passes. The middleware short-circuits on `role === 'superadmin'` without hitting the DB. Permission rows are still seeded for completeness, but they're not consulted.

In-process cache

The `hasPermission` middleware caches per-role permission lookups for **60 seconds**. Means: changing permissions in the DB propagates within a minute. To evict immediately, restart the process.

11. Enrollment Module Deep Dive

src/modules/enrollments/

The dual shape (new vs legacy)

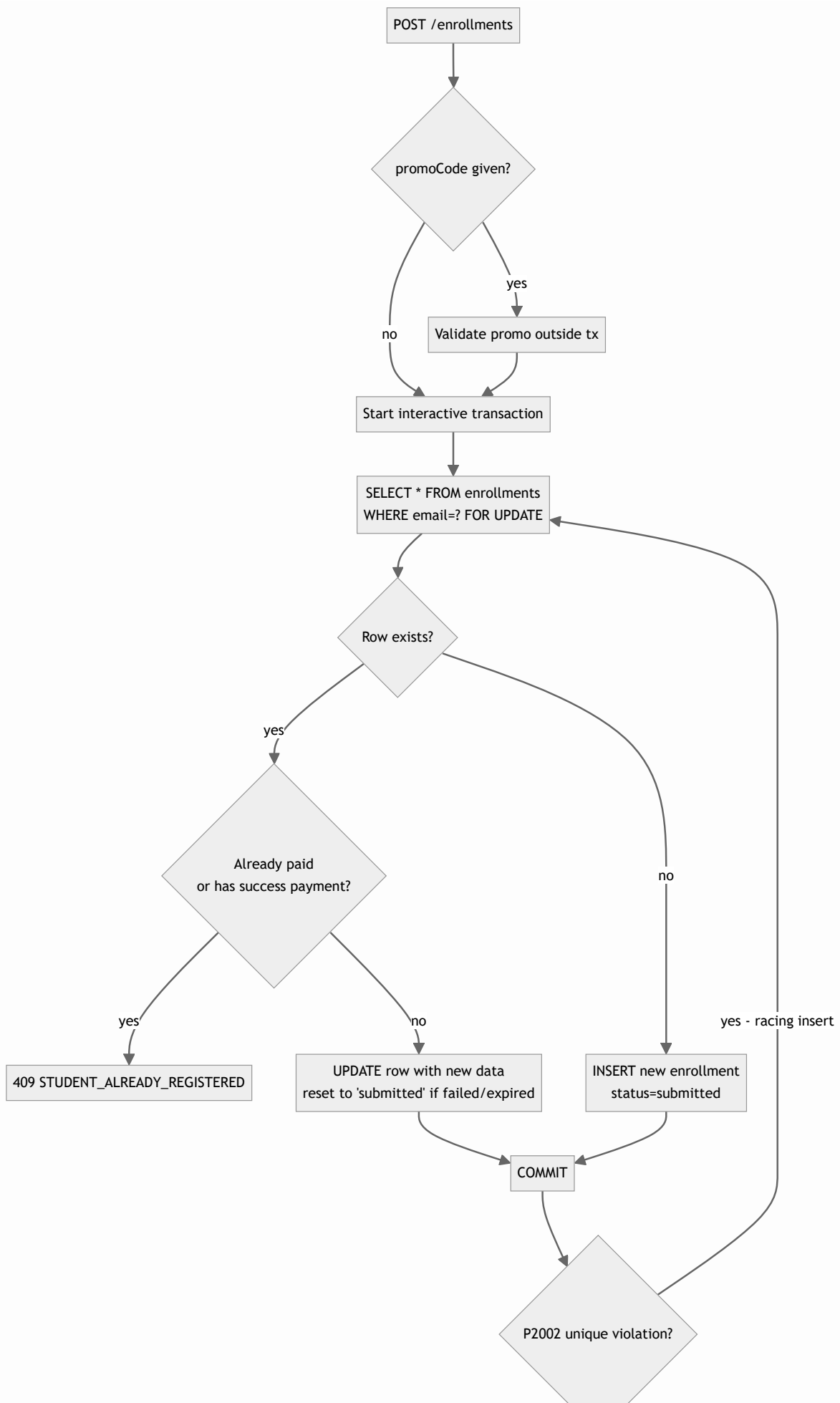
New shape (public form)

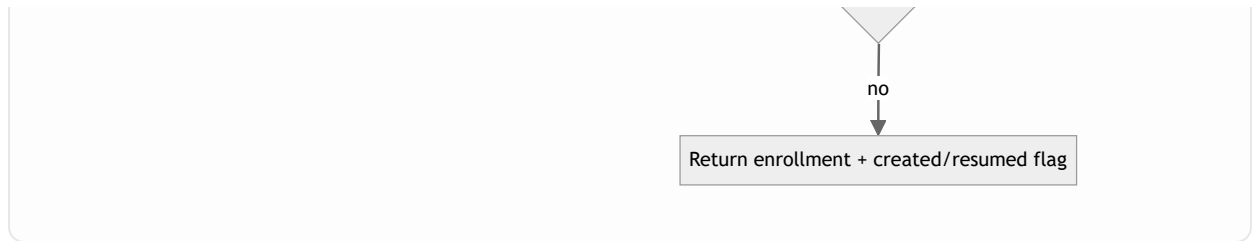
```
{
  "name": "Priya Sharma",
  "email": "priya@example.com",
  "phone": "9876543210",
  "role": "STUDENT",
  "education": "GRADUATE",
  "readiness": "BEGINNER",
  "source": "GOOGLE",
  "promoCode": "EARLY10"
}
```

Legacy shape (Sumago taxonomy)

```
{
  "first_name": "Priya",
  "last_name": "Sharma",
  "email": "priya@example.com",
  "phone_number": "9876543210",
  "plan": "SUMAG030",
  "group": "group_A",
  "unit": "unit_01",
  "phase": "phase_2",
  "segment": "enterprise",
  "amount": 99900
}
```

Race-protected upsert pattern





5-minute dedupe window

Before the transaction, the service checks: did this `(email, phone, status=submitted)` arrive in the last 5 minutes? If yes, return the existing row as-is. Reason: double-click protection — pressing the submit button twice shouldn't create two records.

Enrollment code format

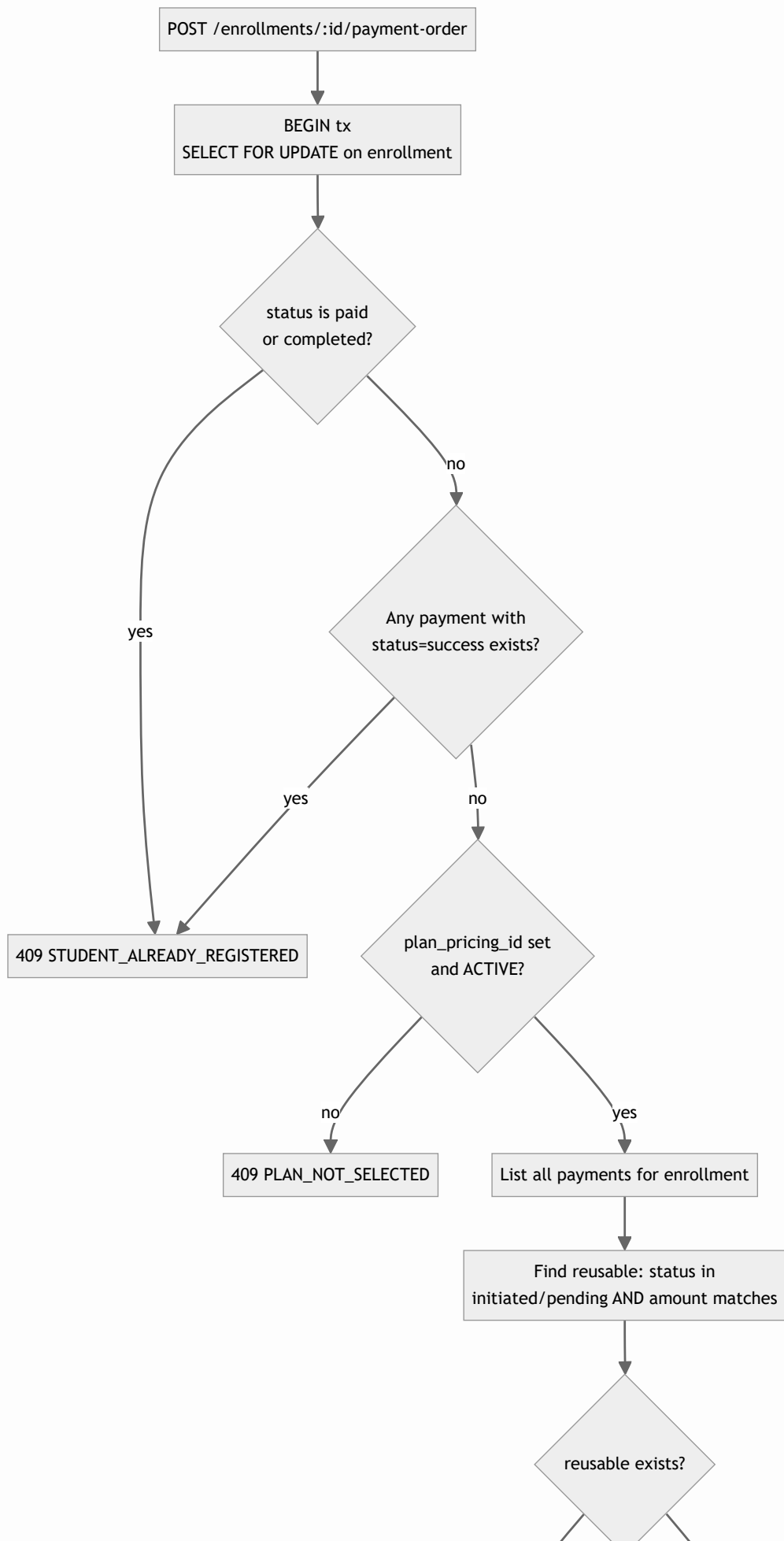
`KS-2605-A1B2C3`

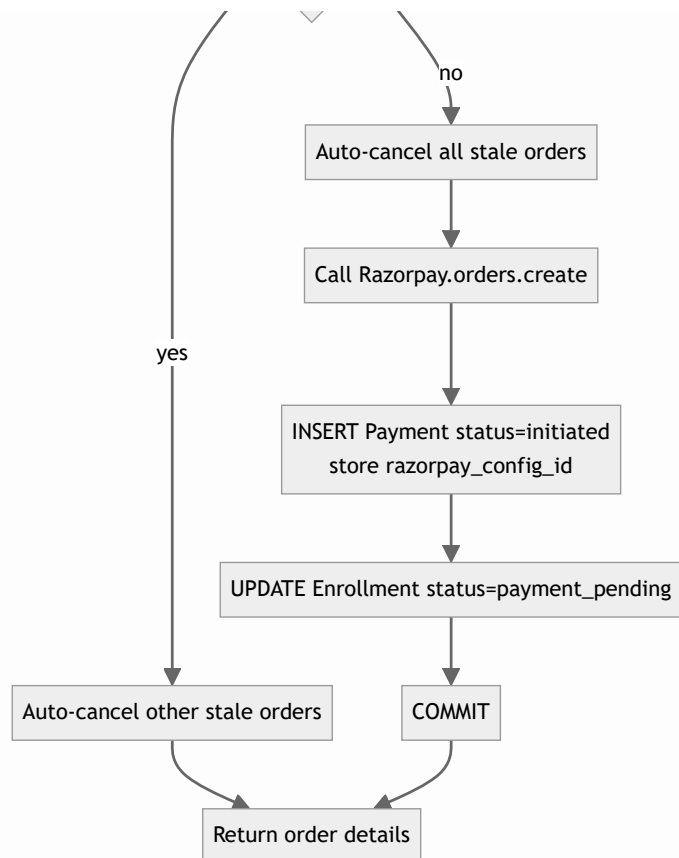
- `KS` — Kommon School prefix (constant)
- `2605` — current UTC year-month (YY + MM)
- `A1B2C3` — 6 random hex chars

12. Payment Module Deep Dive

src/modules/payments/

Order creation flow

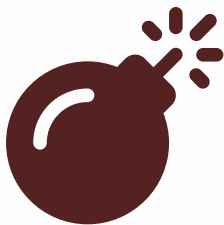




Explain Like I'm 5 (ELI5)

Imagine the student keeps refreshing the payment page. Each refresh hits this endpoint. Without protection, we'd create a fresh Razorpay order every time — confusing for everyone. The "reusable" logic finds an existing initiated order with the right amount and hands it back. Existing stale orders (wrong amount) get cancelled. **At most one live Razorpay order per enrollment at any time.**

Verification flow (race-protected settlement)



Syntax error in text
mermaid version 10.9.6

Why SELECT FOR UPDATE?

Explain Like I'm 5 (ELI5)

Two paths can settle the same payment: (1) the student's browser calling `/verify`, (2) Razorpay's server sending a webhook to us. They could arrive at the exact same millisecond. Without locking, both might:

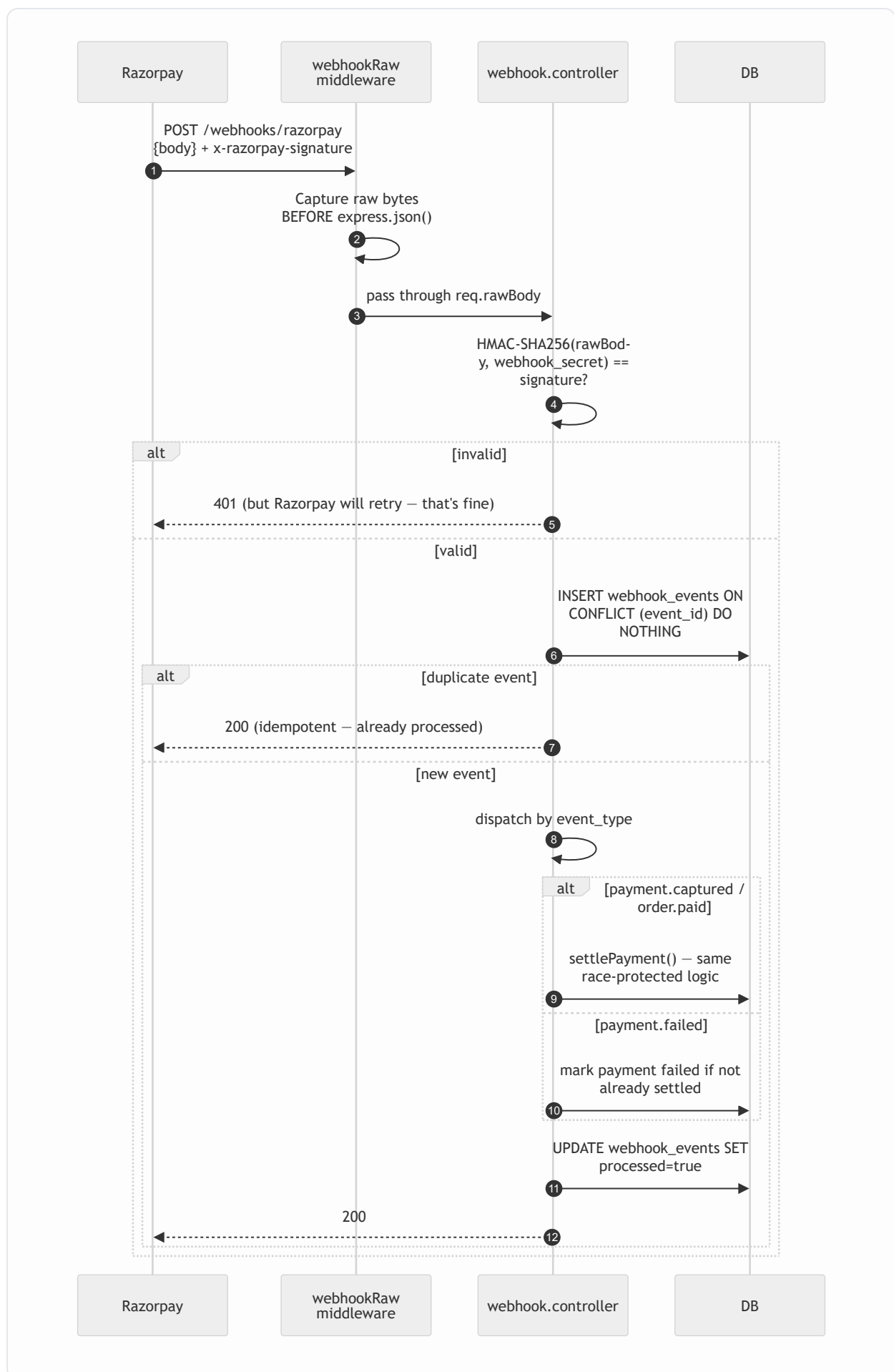
1. Read the Payment row (both see status='initiated').
2. Both update to status='success'.
3. Both enqueue a Sumago sync → two emails to the student!

`SELECT FOR UPDATE` says "the first one to read this row locks it; the second one waits at the door". When the second one finally gets in, it sees status='success' already and returns "alreadySettled" without doing anything. This is the most important pattern in the codebase.

13. Razorpay Webhook Handling

`src/modules/razorpay/webhook.controller.js` + `src/middleware/webhookRaw.middleware.js`

Razorpay independently sends events to `POST /api/v1/webhooks/razorpay` when payments happen. We trust these less than the verify endpoint (because the signature is the only proof), so verification is strict.



Why raw body?

Express normally parses JSON and discards the original bytes. HMAC verification needs the **exact** bytes Razorpay signed — even a single whitespace change breaks the signature. So `webhookRaw` middleware captures the raw buffer before `express.json()` can mangle it. Notice in `app.js` that the webhook route is registered BEFORE `app.use(express.json())`.

Idempotency by event_id

Razorpay retries failed webhooks. The `webhook_events.event_id` column is UNIQUE — a second arrival of the same event_id fails the INSERT, signaling "already processed". We respond 200 immediately without re-processing.

Always 200

Even if internal processing fails, we return 200 to Razorpay. Otherwise they retry. We record the failure in `webhook_events` and a cron (`webhook_retry`) replays it later.

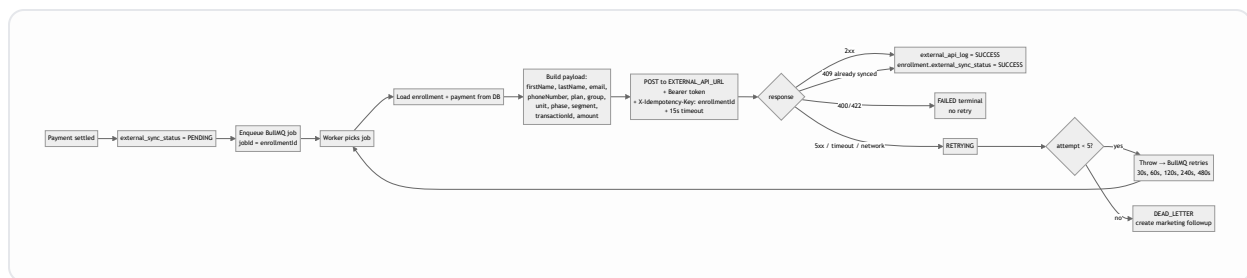
14. External API Sync (Sumago)

`src/modules/externalApi/` + `src/queues/workers/externalApi.worker.js`

What is Sumago?

Sumago is the LMS (Learning Management System) that actually delivers the course. After payment, we have to **provision** the student there — telling Sumago "this person paid, give them access to course X".

The provisioning flow



Circuit breaker (opossum)

Explain Like I'm 5 (ELI5)

If Sumago is clearly down (50% of calls failing), instead of hammering it 100 times in a minute we "open the circuit" — every call fails instantly for 30 seconds. Then we try one to see if it's back. This protects Sumago from a thundering-herd of retries AND saves our worker's time.

Sumago user mirror sync

`src/modules/webhooks/sumagoUserSync.service.js`

We also **pull** user data from Sumago periodically into our `sumago_users` table so the admin UI can list provisioned users without hitting Sumago on every page load.

- Calls `GET /integrations/get-users`

- For each user, computes SHA-256 hash of payload
- Buckets into NEW / CHANGED / UNCHANGED
- Bulk-inserts new, updates changed, touches lastSyncedAt on unchanged

15. BullMQ Queues & Workers

src/queues/

The queue

There's **one queue**: `external-api-sync`. All Sumago provisioning jobs go through it.

Job configuration

Setting	Value	Reason
attempts	5	5 total tries before dead-letter
backoff	exponential, 30s base	30 → 60 → 120 → 240 → 480 seconds
removeOnComplete	keep last 1000	For audit, browseable in BullMQ UI
removeOnFail	false (keep all)	Failed jobs preserved indefinitely
jobId	= enrollmentId	BullMQ deduplicates concurrent enqueues with same id
worker concurrency	5	Process 5 jobs in parallel per worker
rate limiter	50 jobs/sec	Don't overwhelm Sumago

Worker lifecycle events

completed

Log success. `external_api_log` row already marked SUCCESS by the sync function.

failed

- If attempts exhausted: dead-letter + create followup
- Else: log transient failure, BullMQ will retry

Idempotency at the queue level

Explain Like I'm 5 (ELI5)

Imagine you double-click "verify payment". Both clicks enqueue a sync job. With `jobId = enrollmentId`, BullMQ sees the second one and discards it silently (while the first is still waiting or active). Without this, you'd risk calling Sumago twice for the same student.

16. Cron Jobs & Distributed Locks

src/jobs/ + src/utils/distributedLock.js

Why distributed locks?

Explain Like I'm 5 (ELI5)

Say you scale to 3 server instances (instance-A, instance-B, instance-C). Each one has its own cron scheduler. At 2:00 AM, all three try to "expire stale enrollments". Without coordination, you'd do the same work three times. Distributed locks ensure only ONE instance does it. The other two see "lock is held" and skip cleanly.

The lock mechanism

Uses Redis `SET key value NX EX ttl` (atomic set-if-not-exists with expiry). The "value" is a random UUID per acquire. To release, a Lua script checks the value matches before DELETE — so if the lock expired and someone else grabbed it, we don't accidentally release theirs.

The six scheduled jobs

Job	Cron	Lock TTL	What it does
payment_reconciliation	<code>*/5 * * * *</code>	4.5 min	For payments pending > 10 min, ask Razorpay the real status. Settle captured ones. Expire uncaptured after 30 min.
external_api_retry	<code>*/2 * * * *</code>	1.75 min	Reschedule external_api_log rows stuck at <code>retrying</code> (e.g., after Redis restart wiped in-memory BullMQ jobs).
webhook_retry	<code>*/3 * * * *</code>	2.75 min	Replay <code>webhook_events</code> rows with <code>processed=false</code> and attempts < 5.

Job	Cron	Lock TTL	What it does
enrollment_cleanup	0 2 * * *	10 min	Expire submitted enrollments older than 24h.
refresh_token_cleanup	0 3 * * *	10 min	Hard-delete revoked/expired refresh tokens older than 30 days.
followup_reminder	0 9 * * *	5 min	Find followups with next_followup_date <= now and log them grouped by assignee.

17. Admin Modules Overview

Routes prefixed with `/api/v1/admin/` all require Bearer token + appropriate role.

Admin Users (superadmin only)

Method	Path	Purpose
POST	<code>/admin/users</code>	Create admin/marketing user
GET	<code>/admin/users</code>	List with search, filter, sort
PATCH	<code>/admin/users/:id</code>	Update role or password (cannot modify self)
DELETE	<code>/admin/users/:id</code>	Soft-delete + revoke tokens (cannot delete self)

Razorpay Configs (superadmin only)

Multiple configurations can exist; only ONE is active at a time. Secrets stored encrypted with AES-256-GCM using `ENCRYPTION_MASTER_KEY`.

Method	Path	Purpose
POST	<code>/admin/razorpay-configs</code>	Create new config (inactive by default)
PATCH	<code>/admin/razorpay-configs/:id/activate</code>	Atomic activation: deactivate others + activate this
DELETE	<code>/admin/razorpay-configs/:id</code>	Hard-delete (refuses if active)

Dashboard (reports:view)

`GET /admin/dashboard` — Today's enrollment count, today's revenue, pending payments, pending follow-ups. Used by the admin home page.

Admin enrollments — manual & bulk

src/modules/adminEnrollments/

- **Manual:** POST one enrollment with an internal plan + coupon snapshot.
- **Bulk:** Upload CSV. The `candidate_type` auto-sets to `INTERNAL` . Uses multer for file upload + csv-parse.

Reports

Path	Purpose
GET /reports/payments-summary	Counts & totals by status
GET /reports/enrollments-funnel	Count in each EnrollmentStatus stage
GET /reports/external-api-health	Sync health + recent dead-letters
GET /reports/export?type=payments	Streaming CSV download

18. Middleware Stack

Order of execution for a normal API request (from `app.js`):



Each middleware in one line

File	Purpose
<code>requestLogger.middleware.js</code>	Generate traceId, log method+path+status+duration
<code>auth.middleware.js</code>	Parse Bearer header, verify JWT, attach <code>req.user</code>
<code>rbac.middleware.js</code>	<code>authorize([roles])</code> and <code>hasPermission(code)</code>
<code>validate.middleware.js</code>	Run Joi schema against body/params/query, throw 422 if invalid
<code>rateLimit.middleware.js</code>	express-rate-limit per IP — login limiter + global limiter
<code>webhookRaw.middleware.js</code>	Capture raw buffer for HMAC (must be before express.json)
<code>errorHandler.middleware.js</code>	Centralised catch: <code>ApiError</code> → typed envelope, Prisma errors → friendly codes, anything else → 500

19. API Response Pattern

src/utils/ApiResponse.js + src/utils/ApiError.js

Success envelope

```
{
  "success": true,
  "data": { /* the payload */ },
  "message": "optional human message",
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 100,
    "totalPages": 5
  }
}
```

Error envelope

```
{
  "success": false,
  "error": {
    "code": "PLAN_NOT_SELECTED",
    "message": "A subscription plan must be selected before proceeding to payment",
    "details": { /* optional structured info */ }
  },
  "traceId": "9f3e3c1b-..."
}
```

Throwing errors in services

```
const { ApiError } = require('../../utils/ApiError');
const { HTTP, ERROR_CODES } = require('../../config/constants');

if (!enrollment.plan_pricing_id) {
```

```
throw new ApiError(  
  HTTP.CONFLICT,  
  ERROR_CODES.PLAN_NOT_SELECTED,  
  'Pick a plan first',  
  { enrollmentId }  
);  
}
```

The `errorHandler` middleware catches it and emits the standardised envelope automatically.

Pro Tip

Every response carries an `X-Trace-Id` header. When a customer complains about a failed payment, ask for the trace id — you can grep logs across all instances to find the entire request path in seconds.

20. Error Handling Strategy

Five categories

1. **Validation errors** — Joi rejects → 422 with field details
2. **Authorization errors** — Missing/invalid token → 401; insufficient role/permission → 403
3. **Business rule errors** — Thrown as `ApiError` with a code → maps to a specific HTTP status
4. **Prisma errors** — P2002 unique violation, P2025 not found, etc. → friendly codes
5. **Unexpected errors** — Anything else → 500 generic + full log with stack + `traceld`

The graceful degradation principle

- If Redis is down → enqueue fails → fall back to inline HTTP call to Sumago (fire-and-forget).
- If Sumago is down → BullMQ retries → eventually dead-letter → marketing follow-up auto-created.
- If Razorpay webhook signature fails → respond 401 but always 200 on processed-but-errored (let Razorpay retry).
- If audit log write fails → log and continue (never block a business action).

21. Important Edge Cases (Memorize These)

#	Scenario	What happens
1	Student double-clicks "Submit"	5-min dedupe window returns existing row; no duplicate.
2	Student double-clicks "Pay"	<code>SELECT FOR UPDATE</code> serializes; same Razorpay order returned twice.
3	Razorpay sends webhook twice for same event	UNIQUE constraint on <code>event_id</code> + 200 response; no double processing.
4	Verify endpoint & Razorpay webhook arrive at same ms	Whichever locks the Payment row first wins. Loser sees status=success and exits idempotently.
5	Student closes browser after Razorpay but before /verify	The webhook (independent path) will settle the payment.
6	Razorpay webhook never arrives AND /verify never called	<code>payment_reconciliation</code> cron checks Razorpay every 5 min; settles or expires.
7	Sumago returns 5xx	BullMQ retries 5x with backoff. After all fail → DEAD_LETTER + auto-followup.
8	Sumago returns 409 ALREADY_SYNCED	Treated as success — the student is already in Sumago.
9	Sumago returns 400/422 (bad data)	Terminal — no retry, marked FAILED in logs.
10	Redis crashes during work	HTTP server stays up; new payment verifies fall back to inline Sumago call.
11	BullMQ loses jobs after Redis restart	<code>external_api_retry</code> cron finds rows stuck at <code>retrying</code> and reschedules.
12	Multiple instances run cron at same time	Distributed locks via Redis — only one acquires; others log "skipped".
13	Razorpay key rotated mid-flight	Payment row stores <code>razorpay_config_id</code> at order time — verification uses original config.
14	User leaves payment_pending for days	30-min Razorpay timeout → expired status; followup may be auto-created.

#	Scenario	What happens
15	Stolen refresh token reused	Family-based revocation — entire family invalidated, forces re-login.

22. Local Development Setup

Prerequisites

- Node.js 20 or higher
- PostgreSQL 15
- Redis 7
- A Razorpay test account (get keys from dashboard)

Step-by-step

```
# 1. Clone & install
git clone <repo>
cd kommon_school_backend
npm ci

# 2. Set up .env
cp .env.example .env
# Open .env and fill in:
#   DATABASE_URL=postgresql://postgres:postgres@localhost:5432/kommon
#   JWT_ACCESS_SECRET (32+ random chars)
#   JWT_REFRESH_SECRET (32+ random chars)
#   RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET
#   ENCRYPTION_MASTER_KEY: run this to generate:
#     node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
#   SEED_ADMIN_EMAIL=you@example.com
#   SEED_ADMIN_PASSWORD=ChangeMe123!
#   REDIS_URL=redis://localhost:6379

# 3. Start Postgres & Redis
# (Postgres via your OS package manager; Redis: docker run -d -p 6379:6379
redis:7-alpine)

# 4. Run migrations
npx prisma migrate dev

# 5. Seed superadmin + permissions + razorpay config
npm run db:seed

# 6. Start dev server (auto-reload)
```

```
npm run dev
```

```
# 7. Verify
```

```
curl http://localhost:3000/health # should return {"status":"ok"}
```

Smoke test the golden path

```
# Login as superadmin
```

```
curl -X POST http://localhost:3000/api/v1/auth/login \  
  -H 'Content-Type: application/json' \  
  -d '{"email":"you@example.com","password":"ChangeMe123!"}'
```

```
# → save accessToken
```

```
# Create enrollment (no auth needed)
```

```
curl -X POST http://localhost:3000/api/v1/enrollments \  
  -H 'Content-Type: application/json' \  
  -d '{"name":"Test User","email":"test@example.com","phone":"9999999999"}'
```

```
# Pick a plan
```

```
curl -X PATCH http://localhost:3000/api/v1/enrollments/<id>/plan \  
  -H 'Content-Type: application/json' \  
  -d '{"planPricingId":1}'
```

```
# Create Razorpay order
```

```
curl -X POST http://localhost:3000/api/v1/enrollments/<id>/payment-order
```

```
# (use the orderId in the frontend Razorpay checkout to complete payment)
```

Run tests

```
npm test
```

```
# Runs node --test on src/tests/unit/*.test.js
```

```
# Pure in-process, no DB or Redis required
```

23. Glossary of Terms

Term	Meaning
Idempotency	Calling the same operation twice produces the same result as once. Critical for retries and webhooks.
HMAC	Hash-based Message Authentication Code. A way to prove a message came from someone who knows a shared secret. Used by Razorpay signatures.
JWT (JSON Web Token)	A signed token containing claims. Server can verify it without DB lookup.
HS256 / RS256	JWT signing algorithms. HS256 uses one shared secret; RS256 uses public/private key pairs.
Refresh token rotation	Each refresh issues a new refresh token and invalidates the old one. Helps detect token theft.
RBAC	Role-Based Access Control. Users have roles, roles have permissions.
SELECT FOR UPDATE	Postgres feature to lock a row inside a transaction. Other transactions wait until commit/rollback.
Serializable isolation	Strictest Postgres isolation level. Treats concurrent transactions as if they happened one after another.
Race condition	Bug where the outcome depends on which of two concurrent operations finishes first.
Circuit breaker	Pattern that stops calling a failing service for a cooldown period, then probes if it's back.
Exponential backoff	Retry wait doubles each time (30s, 60s, 120s, ...). Avoids hammering a struggling service.
Dead-letter	A job that exhausted all retries. Set aside for manual investigation.
Webhook	An HTTP callback. Service A pushes events to service B's endpoint when things happen.
Cron expression	<code>* * * * *</code> = minute hour day-of-month month day-of-week. <code>* / 5 * * *</code> <code>* </code> = every 5 minutes.

Term	Meaning
Distributed lock	A coordination primitive ensuring only one process holds a "permission slip" at a time, across machines.
Trace ID	UUID attached to every request. Threads through all logs so you can trace one request end-to-end.
Prisma	An ORM — describes the DB schema in one file (<code>schema.prisma</code>) and generates a typed client.
Migration	A versioned SQL change to the DB schema. Prisma generates them when <code>schema.prisma</code> changes.
BullMQ	A Node.js job queue backed by Redis with retries, scheduling, and concurrency control.
PM2	Process manager for Node.js — clustering, log rotation, zero-downtime reload.
Sumago	The external Learning Management System we provision students into after payment.
paise	1/100 of an Indian Rupee. Razorpay (and our amounts) are stored in paise to avoid floats — ₹999 = 99900 paise.